Functional Programming Languages

Chapter 15 — Concepts of Programming Languages

Murat Ak

April 16, 2026

Lecture Notes

What we will cover

Introduction

Mathematical foundations

Fundamentals of FPLs

Lisp and Scheme

Common Lisp

ML

Haskell

F#

Functional features in imperative languages

Comparison and summary

Introduction

Two worldviews of programming

Imperative languages

- Modelled on the von Neumann architecture
- Program = sequence of commands that change memory
- Efficiency is the top priority
- Examples: C, Java, Python

Functional languages

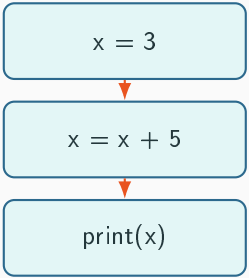
- Modelled on mathematical functions
- Program = evaluation of expressions
- Closer to how humans reason mathematically
- Examples: Lisp, Scheme, Haskell, ML, F#

Key insight

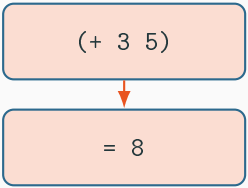
Functional programming shifts the question from *"How do I compute it?"* to *"What is it?"*

Visualizing the two paradigms

Imperative: steps & state



Functional: just evaluate



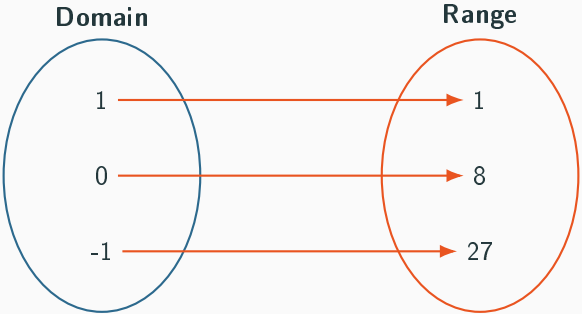
Imperative code *modifies* a variable; functional code *reduces* an expression to a value.

Mathematical foundations

What is a function, mathematically?

Definition

A **mathematical function** is a mapping from members of one set, called the **domain**, to members of another set, called the **range** (or co-domain).




```
cube(x) = x * x * x
```

Every input has exactly one output — the *same* output, every time.

Lambda expressions: nameless functions

A **lambda expression** describes a function without giving it a name. The notation comes from Alonzo Church's **lambda calculus** (1930s).

Math notation:

$$\lambda(x)\ x * x * x$$

is the function `cube(x) = x*x*x`.

Applying it:

$$(\lambda(x)\ x * x * x)(2) \ = \ 8$$

In modern languages:

- Python: `lambda x:  x*x*x`
- JavaScript: `x => x*x*x`
- Scheme: `(lambda (x) (* x x x))`
- Haskell: `\x -> x*x*x`

Why "nameless"?

Lambdas are useful when you need a quick, throw-away function — for example, to pass as an argument to another function.

Functional forms (higher-order functions)

Definition

A **higher-order function** (also: **functional form**) is a function that takes a function as a parameter, returns a function as its result, or both.

Why does this matter?

In math you freely say "the derivative of f". The derivative is an *operator* that takes a function and returns a function. Functional languages treat this as routine.

Two classic examples we will meet:

- Composition — combine two functions into one
- Apply-to-all (a.k.a. `map`) — apply a function to every element of a list

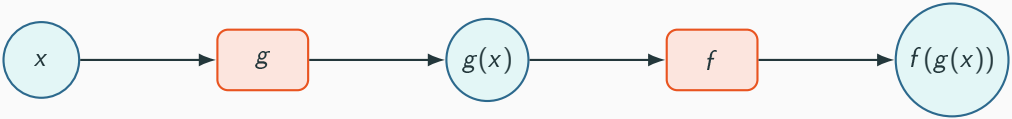
Function composition

If h is the composition of f and g, written $h \equiv f \circ g$, then

$$h(x) \equiv f(g(x))$$

Concrete example

Let $f(x) = x + 2$ and $g(x) = 3x$. Then $(f \circ g)(x) = f(g(x)) = f(3x) = 3x + 2$.



Apply-to-all (map)

Definition

Apply-to-all takes a function and a list, and returns a new list in which that function has been applied to every element.

If $h(x) = x * x$, then

$$\alpha(h,\ (2,3,4)) \ = \ (4,\ 9,\ 16)$$

Python:

```python
list(map(lambda x: x*x, [2,3,4]))
# -> [4, 9, 16]
```

Haskell:

```haskell
map (\x -> x*x) [2,3,4]
-- -> [4,9,16]
```

Fundamentals of FPLs

Fundamentals of FPLs (*Functional Programming Languages*)

- **Goal of an FPL:** mimic mathematical functions as closely as possible.
- **No variables in the imperative sense:** there is nothing to "assign to".
- **No loops:** repetition is expressed with recursion and higher-order functions.
- **Evaluation replaces execution:** you reduce expressions, you do not "run steps".

Consequence

Without mutable variables, programs are much easier to reason about: a piece of code has no hidden state that can surprise you.

Referential transparency

Definition

A language has **referential transparency** when evaluating a function always produces the same result given the same arguments.

Not referentially transparent (Python-like):

```
counter = 0
def f(x):
    global counter
    counter += 1
    return x + counter
# f(3) -> 4, then f(3) -> 5
```

Referentially transparent:

```
f x = x + 1
-- f 3 is ALWAYS 4
```

You can freely replace `f 3` by 4 anywhere in the program — this is why the property is so valuable.

Lisp and Scheme

Lisp (*LISt Processor*): the first FPL

- Created by John McCarthy, **1958** — second-oldest high-level language (after Fortran).
- Originally only two data types:
  - **atoms** — symbols or numbers, e.g. `A`, 42
  - **lists** — parenthesised collections, e.g. `(A B (C D) E)`
- Originally **typeless**.
- Lists are stored internally as singly-linked lists.

Internal representation of the list `(A B C D)`.

Code is data: Lisp's "S-expression" idea

The same parenthesised form can mean *data* or *a function call*:

Interpretation	Meaning of `(A B C)`
as data	a list containing three atoms: A, B, C
as function	apply the function `A` to arguments `B` and `C`

Why it matters

Because code and data share the same form, a Lisp program can easily build other Lisp programs and then run them. This is called **homoiconicity** and is the basis for Lisp's famous macro system.

Scheme: a cleaner Lisp

A mid-1970s dialect of Lisp designed to be simpler and more uniform.

- Uses static (lexical) scoping only.
- Functions are **first-class citizens**:
  - they can be values of expressions,
  - be stored in lists,
  - be assigned to names,
  - be passed as arguments,
  - be returned from other functions.

"First-class citizen" means...

Functions are treated just like any other value (like numbers or strings). You can do everything with a function that you can do with a number.

The Scheme interpreter is a REPL (*Read–Eval–Print Loop*)

- The interpreter repeatedly:
  1. **Reads** one expression,
  2. **Evaluates** it (via a function called `EVAL`),
  3. **Prints** the result,
  4. loops.
- The same pattern is used by Python's `>>` prompt, Ruby's `irb`, Node.js, and many others.
- **Literals evaluate to themselves**: the number 42 evaluates to 42, the string `"hi"` to `"hi"`, etc.

Primitive functions and prefix notation

Scheme uses **prefix notation**: the operator comes *first*, inside the parentheses.

```
(+ 5 2)           ; -> 7
(* 3 4)           ; -> 12
(- 10 (* 2 3))    ; -> 4
(sqrt 16)         ; -> 4
(abs -7)          ; -> 7
```

How primitive function evaluation works:

1. Each argument is evaluated (order unspecified).
2. The values are substituted into the function body.
3. The body is evaluated; the value of the last expression is returned.

Lambda expressions in Scheme

```
(LAMBDA (x) (* x x))              ; nameless "square" function

((LAMBDA (x) (* x x)) 7)          ; -> 49   (applied immediately)

(LAMBDA (a b x) (+ (* a x x) (* b x)))  ; two-parameter example
```

- The names after `LAMBDA` (here `x`, `a`, `b`) are called **bound variables**: they are placeholders inside the function body.
- A lambda can have any number of parameters.
- Lambdas are the foundation; `DEFINE` (next slide) is sugar built on top.

Giving things names: DEFINE

DEFINE is a **special form**: its first argument is *not* evaluated — it is the name being introduced.

Form 1 — bind a name to a value:

```
(DEFINE pi 3.141593)
(DEFINE two_pi (* 2 pi))
```

These are not variables you can reassign — think of Java's `final`.

Form 2 — bind a name to a function (LAMBDA is implicit):

```
(DEFINE (square x) (* x x))
(square 5)   ; -> 25
```

Predicates and booleans

- True is `#T` (or `#t`); false is `#F` (or `#f`).
- Comparison: `=` `<>` `<` `>` `<=` `>=`
- Common predicates end with a question mark: `EVEN?`, `ODD?`, `ZERO?`, `NEGATIVE?`
- Logical negation: `NOT`.

```
(= 3 3)          ; -> #T
(< 2 5)          ; -> #T
(EVEN? 10)       ; -> #T
(ZERO? 0)        ; -> #T
(NOT (< 5 2))    ; -> #T
```

Convention

The `?` suffix for predicates is a Scheme tradition. Ruby borrowed it (`empty?`, `nil?`), Clojure uses it too.

Control flow: IF and COND

IF — binary branch:

```
(IF (<> count 0)
    (/ sum count)
    0)
```

COND — multi-way branch (like a chain of `else if`):

```
(DEFINE (leap? year)
  (COND
    ((ZERO? (MODULO year 400)) #T)
    ((ZERO? (MODULO year 100)) #F)
    (ELSE (ZERO? (MODULO year 4)))))
```

Each clause is `(predicate expression)`; the first true predicate wins.

Working with lists: QUOTE

Because *any* parenthesised form is evaluated by default, we need a way to say "don't evaluate this, I mean it literally". That is **QUOTE**.

```
(QUOTE (A B C))    ; -> (A B C), a list of 3 atoms
'(A B C)           ; shorthand with apostrophe
'(1 2 3)           ; list of three numbers
```

Without QUOTE...

(A B C) would be interpreted as "call function A with arguments B and C" — which is not what you want if you just wanted the list itself.

The classic list trio: CAR, CDR, CONS

Historic names from the original IBM 704 machine:

- CAR (*Contents of the Address part of Register*) — first element (the *head*)
- CDR (*Contents of the Decrement part of Register*) — the rest of the list (the *tail*), pronounced "could-er"
- CONS — "construct": prepend an element to a list

```
(CAR '((A B) C D))    ; -> (A B)
(CDR '((A B) C D))    ; -> (C D)
(CDR '(A))            ; -> ()          the empty list
(CONS 'A '(B C))      ; -> (A B C)
(CONS '(A B) '(C D)) ; -> ((A B) C D)
(CONS 'A 'B)          ; -> (A . B)     a "dotted pair"
```

Building lists: LIST

LIST packages any number of values into a list:

```
(LIST 'apple 'orange 'grape)
; -> (apple orange grape)

(LIST (+ 1 2) (* 3 4) 5)
; -> (3 12 5)      notice: arguments ARE evaluated
```

Contrast with QUOTE: `'(1 2 3)` never evaluates its contents.

Equality predicates: EQ?, EQV?, LIST?, NULL?

EQ? — pointer identity (fast, reliable only for symbols):

```
(EQ? 'A 'A)           ; -> #T
(EQ? 'A '(A B))       ; -> #F
(EQ? 3.4 (+ 3 0.4))   ; -> #T or #F (implementation-defined!)
```

EQV? — value comparison (works for numbers and symbols):

```
(EQV? 3 3)            ; -> #T
(EQV? 3.0 3)          ; -> #F   integer and float differ
```

LIST? — is the argument a list? NULL? — is it the empty list ()?

```
(LIST? '())           ; -> #T
(NULL? '())           ; -> #T
(NULL? '(()))         ; -> #F   a list containing the empty list
```

Example: `member`

"Is the atom `atm` somewhere in the list `a_list`?"

```
(DEFINE (member atm a_list)
  (COND
    ((NULL? a_list) #F)
    ((EQ? atm (CAR a_list)) #T)
    (ELSE (member atm (CDR a_list)))))
```

Reading it out loud

- If the list is empty — the atom is not there: `#F`.
- Else if the head equals the atom — found it: `#T`.
- Else — search the tail.

This is the quintessential functional idiom: no loops, just recursion over head and tail.

Example: `append`

Concatenate two lists:

```
(DEFINE (append list1 list2)
  (COND
    ((NULL? list1) list2)
    (ELSE (CONS (CAR list1)
                (append (CDR list1) list2)))))
```

Trace of `(append '(1 2) '(3 4))`:

```
(CONS 1 (append '(2) '(3 4)))
(CONS 1 (CONS 2 (append '() '(3 4))))
(CONS 1 (CONS 2 '(3 4)))
(CONS 1 '(2 3 4))
'(1 2 3 4)
```

Local bindings: LET

LET introduces local names for a single expression.

```
(LET ((alpha 7))
  (* 5 alpha))        ; -> 35
```

It is actually syntactic sugar for an immediately-applied lambda:

```
((LAMBDA (alpha) (* 5 alpha)) 7)
```

When to use it

Whenever you want to name an intermediate result to make a formula readable — much like a short-lived "local variable" in an imperative language, but without any mutation.

Tail recursion

Definition

A function is **tail recursive** if its recursive call is the very last operation performed — nothing is done with the result afterwards.

Why we care: a compiler can turn a tail-recursive call into a simple jump, giving us the speed of a loop *and* the clarity of recursion. The Scheme standard requires this optimisation.

Not tail recursive (the $*$ waits for the recursive result):

```
(DEFINE (factorial n)
  (IF (<= n 0) 1
      (* n (factorial (- n 1)))))
```

Making `factorial` tail-recursive

We carry the partial result as an extra parameter (an **accumulator**):

```
(DEFINE (facthelper n factpartial)
  (IF (<= n 0)
      factpartial
      (facthelper (- n 1) (* n factpartial))))

(DEFINE (factorial n)
  (facthelper n 1))
```

Now the recursive call is the last thing the function does — the multiplication happens before the call, not after.

Functional forms in Scheme: composition

```
(DEFINE (g x) (* 3 x))
(DEFINE (f x) (+ 2 x))

; General composition: returns a function
(DEFINE (compose f g)
  (LAMBDA (x) (f (g x))))

; Use it:
((compose CAR CDR) '((a b) c d))   ; -> c

; Equivalent to built-in CADDR
(DEFINE (third a_list)
  ((compose CAR (compose CDR CDR)) a_list))
```

Functional forms in Scheme: `map`

```
(DEFINE (map fun a_list)
  (COND
    ((NULL? a_list) '())
    (ELSE (CONS (fun (CAR a_list))
                (map fun (CDR a_list))))))

(map (LAMBDA (n) (* n n n)) '(3 4 2 6))
; -> (27 64 8 216)
```

This is the same "apply-to-all" from the mathematical slides, but implemented in only three lines.

Code that builds code: EVAL

In Lisp, the interpreter itself is just a function, EVAL. We can build a list that *looks like* code and ask EVAL to run it.

```
(DEFINE (adder a_list)
  (COND
    ((NULL? a_list) 0)
    (ELSE (EVAL (CONS '+ a_list)))))

(adder '(1 2 3 4 5))   ; -> 15
```

What happens:

1. (CONS '+ '(1 2 3 4 5)) → (+ 1 2 3 4 5)
2. EVAL runs that list as code → 15.

The '+ quote is essential; without it, + would try to evaluate with no arguments.

Common Lisp

Common Lisp

A 1980s consolidation of the many incompatible Lisp dialects. Large and complex — the opposite of Scheme.

Features included:

- records, arrays, complex numbers, character strings,
- powerful I/O, packages (namespaces) with access control,
- iterative control statements (yes, loops!),
- a rich macro system.

Philosophy

Scheme tries to be minimal and elegant. Common Lisp tries to be big enough to build anything. Both approaches survive today.

Macros and the backquote

Macros run in two steps:

1. *Expand* the macro call into ordinary Lisp code.
2. *Evaluate* the expanded code.

Users define their own macros with `DEFMACRO`.

The **backquote operator** ` is like `QUOTE`, but parts preceded by a comma are "unquoted" (evaluated):

```
`(a (* 3 4) c)        ; -> (a (* 3 4) c)
`(a ,(* 3 4) c)       ; -> (a 12 c)
```

Backquote is the main tool for writing macros that generate code.

ML

ML (*Meta Language*)

A modern functional language very different in spirit from Lisp:

- Syntax closer to Pascal than to Lisp (no ocean of parentheses).
- **Statically scoped**, **strongly typed**, no type coercions.
- Uses **type inference**: the compiler figures out types you didn't write.
- No imperative-style (mutable) variables — identifiers are just names for values.
- Lists, list operations, **pattern matching**, exception handling, and a module system for **ADTs** (*Abstract Data Types*).

ML function declarations

```
fun cube (x : int) = x * x * x;
```

Or with the return type:

```
fun cube (x) : int = x * x * x;
```

If nothing is declared, the compiler defaults to `int` for numbers.

No overloading

ML does not allow user-defined overloaded functions. A `cube` for `real` numbers needs a different name (e.g. `cubeReal`).

Pattern matching in ML

Instead of writing `if–else` chains, ML lets you define a function by one equation per case:

```
fun fact(0) = 1
  | fact(1) = 1
  | fact(n : int) : int = n * fact(n - 1);
```

How it works

When you call `fact(k)`, ML tries the patterns top-down and uses the first that matches `k`.

This style is extremely readable and is the hallmark of modern functional languages (ML, Haskell, F#, Scala, Rust...).

ML lists

```
[3, 5, 7]            (* a list literal     *)
[]                   (* the empty list     *)
4 :: [3, 5, 7]       (* -> [4, 3, 5, 7] -- CONS is :: *)

(* hd = head (CAR), tl = tail (CDR) *)
fun length([]) = 0
  | length(h :: t) = 1 + length(t);

fun append([], lis2) = lis2
  | append(h :: t, lis2) = h :: append(t, lis2);
```

The `h :: t` pattern splits a non-empty list into its head `h` and tail `t` in one step — much tidier than using separate CAR/CDR calls.

Local bindings with `val` and `let`

`val` binds a name to a value — it is not assignment:

```
val distance = time * speed;
```

Re-binding the same name simply hides the old binding; the old value is not mutated.

`let` introduces local bindings for a single expression:

```
let
  val radius = 2.7
  val pi = 3.14159
in
  pi * radius * radius
end;
```

Higher-order functions in ML

`filter` — keep the elements that satisfy a predicate:

```
filter (fn x => x < 100, [25, 1, 711, 50, 100]);
(* -> [25, 1, 50] *)
```

Lambdas use the keyword `fn` (instead of Scheme's `LAMBDA`).

`map` — apply a function to every element:

```
fun cube x = x * x * x;
val cubeList = map cube;
val newList = cubeList [1, 3, 5];
(* -> [1, 27, 125] *)
```

Composition uses the operator `o`:

Currying

Definition

Currying replaces a multi-parameter function with a chain of single-parameter functions, each returning the next. Named after the logician Haskell Curry.

```
(* Uncurried: takes a tuple *)
fun addPair (a, b) = a + b;
addPair(3, 5);          (* -> 8 *)

(* Curried: no commas between parameters *)
fun add a b = a + b;
add 3 5;                (* -> 8 *)
```

The curried `add` is really a function that takes `a` and returns a *new function* that takes `b`. This unlocks a powerful trick...

Partial evaluation (partial application)

Because a curried function returns a function when you give it fewer arguments, we can pre-supply some arguments:

```
fun add a b = a + b;

val add5 = add 5;     (* supply just 'a' *)
(* add5 is now: fn b => 5 + b *)

val num = add5 10;    (* -> 15 *)
```

Why is this useful?

You can build a "family" of specialised functions from a general one — no boilerplate, no wrapper classes.

Haskell

Haskell

Named after Haskell Curry. Shares much with ML — syntax style, static scoping, strong types, type inference, pattern matching — but is purely functional:

- No variables you can modify.
- No assignment statements.
- No side effects of any kind in normal code.

"Purely functional" in practice

I/O, random numbers, mutable state — all these exist in Haskell, but they are kept strictly separate from pure computation via a mechanism called **monads**. This is beyond our scope, but the idea is powerful: you can always *see* from the type whether a function has side effects.

Haskell syntax: pattern matching & guards

Pattern matching by equations (very ML-like, but no commas, no `fun`):

```
fact 0 = 1
fact 1 = 1
fact n = n * fact (n - 1)
```

Guards (vertical bars) give multi-way conditions per equation:

```
sub n
  | n < 10     = 0
  | n > 100    = 2
  | otherwise = 1
```

Polymorphism comes for free via type inference:

Haskell lists

```
directions = ["north", "south", "east", "west"]
length directions       -- 4       (note: # is older syntax)

[2, 4..10]              -- [2,4,6,8,10]    arithmetic series
[1, 3] ++ [5, 7]        -- [1,3,5,7]       concatenation

1 : [3, 5, 7]           -- [1,3,5,7]       CONS = :
```

Pattern parameters in action — product of a list:

```
product []      = 1
product (a:x)   = a * product x

fact n = product [1..n]
```

List comprehensions

A list comprehension builds a list by describing its elements, just like set-builder notation in math: $\{n^3 \mid n \in \{1, \ldots, 50\}\}$.

```
[n * n * n | n <- [1..50]]
-- [1, 8, 27, 64, 125, ..., 125000]
```

With a **test** (filter):

```
factors n = [i | i <- [1 .. n `div` 2], n `mod` i == 0]
```

Backticks turn a normal function into an infix operator (`n `mod` i` is the same as `mod n i`).

Python borrowed this!

Python's `[x*x for x in range(10) if x%2==0]` is directly inspired by Haskell list comprehensions.

Quicksort in Haskell — the famous one-pager

```haskell
sort []     = []
sort (h:t) =
    sort [b | b <- t, b <= h]
    ++ [h] ++
    sort [b | b <- t, b >  h]
```

Read it as:

- Sorting the empty list gives the empty list.
- Otherwise, pick the head `h`; sort everything in the tail that is $\leq h$, then `h` itself, then the sorted rest.

Hard to beat that for clarity — compare this to 20+ lines of C!

Lazy evaluation

Definitions

- A language is **strict** if arguments are fully evaluated *before* a function is called.
- A language is **non-strict** (or uses **lazy evaluation**) if values are computed only when they are actually needed.

Haskell is lazy. One consequence: infinite data structures are fine.

```
positives = [0..]                      -- all natural numbers
squares   = [n * n | n <- [0..]]       -- all perfect squares

-- Does the list 'squares' contain 16?
member []     b = False
member (a:x) b = (a == b) || member x b

member squares 16   -- -> True
```

A smarter `member`

The previous `member` only works when the item *is* in the list — otherwise it loops forever over an infinite list.

If the list is sorted in increasing order, we can stop early:

```
member2 n (m:x)
  | m <  n     = member2 n x    -- skip past
  | m == n     = True           -- found
  | otherwise = False          -- overshot; stop
```

Combined with lazy evaluation, this terminates on infinite sorted lists. Laziness + purity is what makes tricks like this safe.

F#

F#

A .NET functional language based on OCaml (a descendant of ML, influenced by Haskell).

- Primarily functional, but also supports imperative and object-oriented programming.
- Full-featured **IDE** (*Integrated Development Environment*), huge utility library, interoperates with other .NET languages (C#, VB.NET).
- Includes tuples, lists, **discriminated unions**, records, and both mutable and immutable arrays.
- Supports generic sequences generated lazily.

F# sequences (seq)

A **sequence** is a lazy stream of values — the F# analogue of Haskell lists.

```
let x = seq {1..4};;            // 1,2,3,4

let y = seq {0..10000000};;     // computed lazily!

// Custom step size
let seq1 = seq {1..2..7}        // 1,3,5,7

// Comprehension-like generator
let cubes = seq {for i in 1..4 -> (i, i * i * i)};;
// (1,1); (2,8); (3,27); (4,64)
```

The pipeline operator |>

|> sends its left operand as the *last* argument to the function on its right. This lets you read data flows left-to-right, top-to-bottom:

```
let myNums = [1; 2; 3; 4; 5]

let evensTimesFive =
    myNums
    |> List.filter (fun n -> n % 2 = 0)
    |> List.map    (fun n -> 5 * n)

// result: [10; 20]
```

Familiar?

Unix shell pipes (`cat file | grep foo | sort`) use the same idea. So do JavaScript's `.filter().map()` chains.

Composition and curried functions in F#

Composition uses »:

```
// (f >> g) x   is  g(f(x))
let addOne x = x + 1
let double  x = x * 2
let addOneThenDouble = addOne >> double
addOneThenDouble 3     // -> 8
```

Curried functions work as in ML:

```
let add a b = a + b;;
let add5 = add 5;;          // partial application
add5 10                     // -> 15
```

Functional features in imperative languages

FP features leak into mainstream languages

Modern "imperative" languages now borrow heavily from FP.

Anonymous functions (lambdas) everywhere:

```javascript
// JavaScript
const square = x => x * x;
```

```csharp
// C#
Func<int,bool> isEven = i => (i % 2) == 0;
```

```python
# Python
f = lambda a, b: 2 * a - b
```

Higher-order functions in Python and Ruby

Python has `map`, `filter`, and partial application:

```
list(map(lambda x: x**3, [2, 4, 6, 8]))
# -> [8, 64, 216, 512]

from functools import partial
from operator  import add
add5 = partial(add, 5)
add5(15)          # -> 20
```

Ruby has **blocks**, which can become lambdas and be curried:

```
times  = lambda {|a, b| a * b}
times.(3, 4)                  # -> 12

times5 = times.curry.(5)
times5.(3)                    # -> 15
```

Comparison and summary

Functional vs imperative: the trade-offs

Imperative	Functional
Efficient execution	Less efficient (traditionally)
Complex semantics	Simple semantics (substitution of equals for equals)
Complex syntax	Simple syntax
Concurrency is programmer-designed	Programs can often be parallelised automatically
Mutable state everywhere	Referential transparency

The convergence

The line is blurring. Imperative languages add lambdas, `map`, immutability. Functional languages add efficient compilers, IDEs, and OOP features. Pick the right tool for each job.

Summary

- FPLs are built on **function application**, **conditional expressions**, **recursion**, and **functional forms** (higher-order functions).
- **Lisp** began purely functional and later added imperative features.
- **Scheme** is a small, clean Lisp dialect with static scoping.
- **Common Lisp** is a large, industrial-strength Lisp.
- **ML** is statically typed with type inference and pattern matching.
- **Haskell** is lazy, pure, and supports infinite data structures and list comprehensions.
- **F#** brings functional programming to the .NET ecosystem alongside OOP and imperative features.
- **Mainstream languages** now include many FP features.
- Pure FP has real advantages (clarity, parallelism, reasoning) but is still less widespread than imperative programming.

Questions?